LeetCode HOT 100

PART 16

Multidimensional Dynamic Programming

多维动态规划

91. 不同路径 (No.62) MEDIUM

一个机器人位于一个 `m x n` 网格的左上角 （起始点在下图中标记为 “Start” ）。

机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为 “Finish” ）。

问总共有多少条不同的路径?

示例 1：

> **输入：** m = 3, n = 7
> **输出：** 28

示例 2：

> **输入：** m = 3, n = 2
> **输出：** 3
> **解释：**
> 从左上角开始，总共有 3 条路径可以到达右下角。
> 1. 向右 -> 向下 -> 向下
> 2. 向下 -> 向下 -> 向右
> 3. 向下 -> 向右 -> 向下

示例 3：

> **输入：** m = 7, n = 3
> **输出：** 28

示例 4：

> **输入：** m = 3, n = 3
> **输出：** 6

提示：

- `1 <= m, n <= 100`
- 题目数据保证答案小于等于 $2 * 10^9$

91. 不同路径 (No.62) MEDIUM

思路

终点所在行和列，走法都只有1种，因为只能向右或向下移动。而中间所有的网格点的走法，可以一步向下，加上下方网格的走法；也可以一步向右，加上右方网格的走法。

实际运行时，把整个网格180°翻转，终点在f[0][0]，所求为f[m-1][n-1]起点处的走法。状态转移方程为：

$$f[i][j] = f[i-1][j] + f[i][j-1]$$

可以看到f[i][j]只依赖i或j前1期的值，因此可以使用滚动数组进行状态压缩，变成一维动态规划。用f[j]存储即可，由于原先f[i-1][j]为f[i][j]更新前的值，因此状态转移方程变成：f[j] = f[j] + f[j-1]，即f[j] += f[j-1]。

当然这道题其实可以通过排列组合直接在$\mathcal{O}(1)$的时间内求得C_{m+n-2}^{m-1}。

Solution 1 – 优化前

```python
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = f[i-1][j] + f[i][j-1]
        return f[m-1][n-1]
```

时间复杂度：$\mathcal{O}(mn)$；**空间复杂度**：$\mathcal{O}(mn)$

Solution 2 – 优化后

```python
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [1] * n
        for _ in range(1, m):
            for j in range(1, n):
                f[j] += f[j-1]
        return f[n-1]
```

时间复杂度：$\mathcal{O}(mn)$；**空间复杂度**：$\mathcal{O}(n)$

92. 最小路径和 (No.64) MEDIUM

给定一个包含非负整数的 `m x n` 网格 `grid` ，请找出一条从左上角到右下角的路径，使得路径上的数字总和为最小。

说明： 每次只能向下或者向右移动一步。

示例 1：

1	3	1
1	5	1
4	2	1

> **输入：** `grid = [[1,3,1],[1,5,1],[4,2,1]]`
> **输出：** `7`
> **解释：** 因为路径 1→3→1→1→1 的总和最小。

示例 2：

> **输入：** `grid = [[1,2,3],[4,5,6]]`
> **输出：** `12`

提示：

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 200`
- `0 <= grid[i][j] <= 200`

92. 最小路径和 (No.64) MEDIUM

思路

f[i][j]定义为(0,0)到(i,j)的最小路径和。初始化第一行和第一列，最小路径和只能从(0,0)闷头往下或者往右，因此直接对grid第一行和第一列不断求和可得。其他区域的最小路径可能来自于其上方，也可能来自于其左方，二者取小加上本格的数值即可。状态转移方程为f[i][j] = min(f[i-1][j],f[i][j-1]) + grid[i][j]。

时间复杂度：$\mathcal{O}(mn)$

空间复杂度：$\mathcal{O}(mn)$

```python
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        # f[i][j]表示(0,0)到(i,j)的最小路径和
        f = [[0] * n for _ in range(m)]
        # 初始化第一行和第一列
        f[0][0] = grid[0][0]
        for i in range(1, m):
            f[i][0] = f[i-1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j-1] + grid[0][j]
        # 其他区域DP
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j]
        return f[m-1][n-1]
```

93. 最长回文子串 (No.5) MEDIUM

给你一个字符串 `s`，找到 `s` 中最长的 回文 子串。

示例 1：

> **输入：** `s = "babad"`
> **输出：** `"bab"`
> **解释：** `"aba"` 同样是符合题意的答案。

示例 2：

> **输入：** `s = "cbbd"`
> **输出：** `"bb"`

提示：

- `1 <= s.length <= 1000`
- `s` 仅由数字和英文字母组成

93. 最长回文子串 (No.5) MEDIUM

Solution 1 – 动态规划

思路

f[i][j]定义为s[i:j+1]是否是回文子串。枚举时，步长/子串长度step从1遍历到n-1，然后枚举左边界i，则右边界j=i+step，从而根据回文子串定义，状态转移方程为f[i][j] = f[i+1][j-1] AND s[i]==s[j]，里面回文，最外面一层相等。当出现回文子串时，判断长度是否需要更新，如需更新，同步更新最长回文子串的左右边界。

可以看到f[i][j]依赖其内侧i+1,j-1的状态，因此遍历顺序必须由内而外，所以要从长度为1的子串开始遍历，不断扩展长度。

初始化True有两方面目的：①f[i][i]=True，单个字符必回文；②f[i][i-1]=True，空串必回文。

时间复杂度：$\mathcal{O}(n^2)$

空间复杂度：$\mathcal{O}(n^2)$

```python
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        # f[i][j]表示s[i:j+1]是否是回文子串
        f = [[True] * n for _ in range(n)]  # 初始化f[i][i]和f[i][i-1]为True
        # 记录最长回文子串信息
        max_i, max_j = 0, 0
        for step in range(1, n):  # 步长从1增大到n-1
            for i in range(n-step):
                j = i + step
                f[i][j] = f[i+1][j-1] and s[i] == s[j]
                if f[i][j] and step+1 > max_j - max_i + 1:  # 更新最长回文子串信息
                    max_i, max_j = i, j
        return s[max_i:max_j+1]
```

93. 最长回文子串 (No.5) MEDIUM

Solution 2 – 双指针

思路

中心扩展法：枚举所有中心点，用双指针从中心往两侧扩展，从而枚举出所有的回文子串。中心点可能有1个，可能有2个，统一的做法是枚举所有2倍距离的点，让i从0遍历到2n-1，left=i//2,right=(i+1)//2，就能枚举出长度n内所有中心点的情况。left和right从中心往两侧扩展跳出循环时，left+1到right-1的位置就是回文子串。

时间复杂度：$\mathcal{O}(n^2)$

空间复杂度：$\mathcal{O}(1)$

```python
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        max_left, max_right = 0, 0
        for i in range(2 * n - 1):
            # 通过i枚举所有中心点（可能是1个点，可能是2个点）
            left, right = i // 2, (i + 1) // 2
            while left >= 0 and right < n and s[left] == s[right]:
                left -= 1
                right += 1
            # 此时s[l+1:r]是回文子串
            if right - left - 1 > max_right - max_left + 1:
                max_left, max_right = left + 1, right - 1
        return s[max_left:max_right+1]
```

94. 最长公共子序列 (No.1143) MEDIUM

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长 **公共子序列** 的长度。如果不存在 **公共子序列** ，返回 `0` 。

一个字符串的 **子序列** 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

- 例如，`"ace"` 是 `"abcde"` 的子序列，但 `"aec"` 不是 `"abcde"` 的子序列。

两个字符串的 **公共子序列** 是这两个字符串所共同拥有的子序列。

示例 1：

> **输入：** `text1 = "abcde", text2 = "ace"`
> **输出：** `3`
> **解释：** 最长公共子序列是 `"ace"` ，它的长度为 `3` 。

示例 2：

> **输入：** `text1 = "abc", text2 = "abc"`
> **输出：** `3`
> **解释：** 最长公共子序列是 `"abc"` ，它的长度为 `3` 。

示例 3：

> **输入：** `text1 = "abc", text2 = "def"`
> **输出：** `0`
> **解释：** 两个字符串没有公共子序列，返回 `0` 。

提示：

- `1 <= text1.length, text2.length <= 1000`
- `text1` 和 `text2` 仅由小写英文字符组成。

94. 最长公共子序列 (No.1143) MEDIUM

思路

非常经典的多维DP题，f[i][j]表示text1[0:i]和text2[0:j]的LCS长度，i和j都是位数，因此遍历到i和j时对应字符是text1[i-1]和text2[j-1]。i或j=0对应空串，空串与任意字符串的LCS长度都是0。状态转移方程：

$$f[i][j] = \begin{cases} f[i-1][j-1]+1, & text1[i-1] = text2[j-1] \\ \max(f[i][j-1], f[i-1][j]), & text1[i-1] \neq text2[j-1] \end{cases}$$

当出现两个新字符相同，那LCS长度必然在原先都没出现过的基础上+1（当然此时可能和f[i][j-1],f[i-1][j]相等）；当出现两个新字符不相同，那么LCS长度是少掉任意一个新字符的结果取大，因为新字符并不会增加LCS长度。

时间复杂度：$\mathcal{O}(mn)$

空间复杂度：$\mathcal{O}(mn)$

```python
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        # f[i][j]表示text1[0:i]和text2[0:j]的最长公共子序列长度
        f = [[0] * (n+2) for _ in range(m+2)]  # 初始化所有含有空串情况为0
        for i in range(1, m+1):
            for j in range(1, n+1):  # 注意下标对应，ij是位数，下标要-1
                if text1[i-1] == text2[j-1]:  # 新字符相等，原先最长长度+1
                    f[i][j] = f[i-1][j-1] + 1
                else:  # 新字符不相等，和少掉其中一个字符的结果一致
                    f[i][j] = max(f[i-1][j], f[i][j-1])
        return f[m][n]
```

95. 编辑距离 (No.72) MEDIUM

给你两个单词 `word1` 和 `word2`， *请返回将 `word1` 转换成 `word2` 所使用的最少操作数* 。

你可以对一个单词进行如下三种操作：

- 插入一个字符
- 删除一个字符
- 替换一个字符

示例 1：

> **输入：** word1 = "horse", word2 = "ros"
> **输出：** 3
> **解释：**
> horse -> rorse (将 'h' 替换为 'r')
> rorse -> rose (删除 'r')
> rose -> ros (删除 'e')

示例 2：

> **输入：** word1 = "intention", word2 = "execution"
> **输出：** 5
> **解释：**
> intention -> inention (删除 't')
> inention -> enention (将 'i' 替换为 'e')
> enention -> exention (将 'n' 替换为 'x')
> exention -> exection (将 'n' 替换为 'c')
> exection -> execution (插入 'u')

提示：

- `0 <= word1.length, word2.length <= 500`
- `word1` 和 `word2` 由小写英文字母组成

95. 编辑距离 (No.72) MEDIUM

思路

也是非常经典的多维DP题，f[i][j]表示text1[0:i]和text2[0:j]的最短编辑距离，i和j都是位数，因此遍历到i和j时对应字符是word1[i-1]和word2[j-1]。初始化i或j=0对应空串的情况，空串与长度为k的字符串的编辑距离是k。状态转移方程：

$$f[i][j] = \begin{cases} f[i-1][j-1], & word1[i-1] = word2[j-1] \\ \min(f[i][j-1], f[i-1][j], f[i-1][j-1]) + 1, & word1[i-1] \neq word2[j-1] \end{cases}$$

当出现两个新字符相同，最短编辑距离和原先没有两个字符时一致（当然此时可能和f[i][j-1],f[i-1][j]的最短编辑距离相等）；当出现两个新字符不相同，那么编辑距离是进行一步增删改回退到之前状态的结果+1，其实动态规划里很多这种只考虑一步转移的思想。

时间复杂度：$\mathcal{O}(mn)$

空间复杂度：$\mathcal{O}(mn)$

```python
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        # f[i][j]表示word1[0:i]和word2[0:j]的最短编辑距离
        f = [[0] * (n+1) for _ in range(m+1)]
        # 初始化空串相关的编辑距离
        for i in range(m+1):
            f[i][0] = i
        for j in range(n+1):
            f[0][j] = j
        for i in range(1, m+1):
            for j in range(1, n+1):
                if word1[i-1] == word2[j-1]:  # 新字符相同不用改
                    f[i][j] = f[i-1][j-1]
                else:  # 新字符不同取增删改操作里面的最小值
                    f[i][j] = min(f[i-1][j], f[i][j-1], f[i-1][j-1]) + 1
        return f[m][n]
```